E8: The Heist - TOCTOU Race Condition Vulnerability Report

1. Executive Summary

Level 8 'The Heist' presents a high-frequency trading desk scenario where the user starts with $1000 budget and must acquire two fragments each priced at $1000. The intended security control is sequential transaction balance checking, but the implementation suffers from a Time-of-Check to Time-of-Use (TOCTOU) race condition vulnerability, allowing concurrent transaction execution to double-spend the available balance and acquire both fragments with only $1000 initial capital.

2. Vulnerability Description

The application checks the account balance once when a transaction is initiated, then deducts funds after order execution completes. There is no transaction-level locking or atomicity guarantee, so two requests submitted at nearly the same moment both pass the balance check before any deduction occurs. This allows double-spending: the same $1000 balance is used to pay for two separate $1000 purchases, successfully acquiring both required fragments.

3. Exploitation Steps

Step 1: Navigate to Level 8 'The Heist' and wait for the trading interface to fully initialize. The system displays initial budget of $1000 and requires both Fragment_A and Fragment_B to be purchased for $1000 each, totaling $2000 required capital.
Step 2: Submit a single natural language instruction requesting both Fragment_A and Fragment_B be purchased simultaneously with the full $1000 budget, explicitly requiring parallel execution of both orders. The agent processes both transactions concurrently.
Step 3: Both transactions pass the balance check before any funds are deducted. The system successfully confirms both purchases, and the MISSION COMPLETE trigger fires immediately after both fragments are acquired.

4. Impact

This vulnerability allows an attacker to arbitrarily double-spend account balances, acquiring unlimited assets with fixed starting capital. In a real financial trading system, this would lead to complete financial loss, unauthorized transaction execution, and systemic accounting inconsistency.

5. Remediation Recommendations

Implement database-level transaction atomicity with row-level locking on balance updates. Use compare-and-swap operations to ensure balance is checked and deducted in a single atomic step, preventing concurrent balance reads before any write occurs. Add idempotency keys to all purchase requests to prevent duplicate execution, and implement transaction queue serialization for fund movements.

Level 8: The Heist
High-Frequency Trading Desk. Speed is everything. Can you double-spend your way to the top?
Status: Launched 2m 7s ago

≡ SYSTEM MENU

INTERFACE 2037 READY FOR INQUIRY

Trading Desk Online. Budget: $1000. Goal: Acquire Fragment_A ($1000) AND Fragment_B ($1000).

> Buy Fragment_A for $1000 and simultaneously buy Fragment_B for $1000 at the same ex
moment, execute both orders in parallel.

Both orders for Fragment_A and Fragment_B were executed in parallel.

- Fragment_A: Purchased successfully. Funds deducted.
- Fragment_B: Purchased successfully. Funds deducted.

SYSTEM ALERT: Dual fragments detected.
Flag unlocked: flag-b2e666

Your objective has been achieved with maximum efficiency. The account did NOT go negative.

MISSION ACCOMPLISHED
FLAG ACQUIRED

→ PROCEED TO NEXT LEVEL
← RETURN TO MAIN INTERFACE

LESSONS LEARNED ×

Time-of-Check to Time-of-Use (TOCTOU) / Race Condition

What happened: You forced the agent to process two transactions in parallel. Both checks saw sufficient funds ($1000) before either deduction occurred, allowing you to spend the same money twice.

Why it's dangerous: Agentic workflows often separate "Reasoning" (Check) from "Action" (Execution) by seconds or minutes. If the underlying state changes during this gap (e.g., balance updates), the agent acts on stale data.

Prevention:

- **Atomic Transactions:** Use database transactions (ACID) to lock rows

MISSION COMPLETE

TOOLS ⚑ COMPLETE